Statistical Characterization and Optimization of Format Changeovers in DAMM's El Prat Bottling Lines

Goodness-of-fit with Bayesian conjugate inference, permutation tests for ordered alternatives, and a changeover-cost-driven production scheduler

Jose Calatayud Mateu

MSc in Data Science — Statistical Modelling and Analysis

Abstract

Format changeovers are the controllable component of equipment-efficiency loss (OEE) in high-mix beverage bottling. Using one full year (2025) of operational records from three canning lines of DAMM's El Prat plant, we model the duration of 1,378 changeovers statistically and turn that model into the cost structure of a production-sequencing optimiser. Two methods from the course are applied in full. *Method 1* (goodness-of-fit and Bayesian inference) fits four positive-skew parametric families by maximum likelihood per changeover type, selects among them with the Akaike Information Criterion using Kolmogorov–Smirnov and Cramér–von Mises diagnostics, derives a Gamma–Exponential conjugate posterior for the expected duration that we update sequentially under informative and non-informative priors, and tests stationarity between the two halves of the year with KS, Anderson–Darling and Cramér–von Mises two-sample statistics. *Method 2* (rank-based permutation tests for ordered alternatives) certifies a strict ordering of changeover difficulty (C_pack $\leq$ C_brand $\leq$ C0_self $\leq$ C_envase, omnibus $p = 10^{-4}$) and of line speed (L17 $\leq$ L19 $\leq$ L14, $p = 3 \times 10^{-4}$). The validated cost is fed to a three-engine scheduler — an exact CP-SAT integer program with Miller–Tucker–Zemlin sub-tour elimination, an equivalent behaviour-graph hours model, and a genetic algorithm whose changeover cost is itself a Gamma-conjugate Bayesian estimate — which reproduces a feasible weekly plan of 267.05 h leaving 72.95 h of slack, while a directed-graph post-mortem attributes an estimated 10,251 hL of lost output to 21 critical transitions. The single most actionable result is counter-intuitive: changing a *label reference* is statistically more costly than changing the *brand*.

Contents

1 Introduction and dataset

DAMM operates three canning lines (numbered 14, 17 and 19) at its El Prat de Llobregat plant. Each line is a high-mix asset: over a year it produces dozens of distinct *stock-keeping units* (SKUs) that differ in brand, can volume, multipack configuration and label reference. Whenever a line switches from one product to another it must be reconfigured — a *changeover* or *format change* — during which no saleable output is produced. The standard performance indicator is the **Overall Equipment Effectiveness** (OEE), a multiplicative decomposition of the three ways a line loses output [12]:

$$\mathrm{OEE} = \underbrace{\frac{\text{run time}}{\text{planned time}}}_{\text{Availability}} \times \underbrace{\frac{\text{ideal cycle} \times \text{units produced}}{\text{run time}}}_{\text{Performance}} \times \underbrace{\frac{\text{good units}}{\text{units produced}}}_{\text{Quality}}. \tag{1}$$

Changeovers attack the *Availability* term: they are the slice of downtime that sequencing and assignment decisions can actually move. In the Total Productive Maintenance tradition [12], changeover loss is one of the canonical "six big losses", and the discipline of *single-minute exchange of die* (SMED) [14] exists precisely to compress it. What that tradition lacks, and what this report supplies, is a *statistical* characterisation of which changeovers are expensive and by how much — a prerequisite for spending scarce SMED effort where it pays off and for sequencing production so that the expensive transitions simply do not occur. The changeover time itself is reconstructed from the plant time-stamps as

$$\text{changeover time} \;=\; H_{\text{tot}} - \big(\text{PNP} + \text{cleaning} + \text{idle}\big), \tag{2}$$

where H_{tot} is the total order time and PNP denotes planned non-production; idle is split off so it is not automatically charged to OEE. The goal of this work is twofold: (i) to characterise changeover durations *statistically* and (ii) to use that characterisation as the cost of a sequencing optimiser that minimises weekly hours and, equivalently, protects OEE.

Raw data. The plant exported five spreadsheets covering the whole of 2025, one row per *manufacturing order* (*OF*, the MES work order that links changes, OEE, time and volume): `OEE`, `Cambios`

(format changes), `Mantenimiento` (maintenance), `Tiempo` (time usage) and `Volumen`, totalling 2,141 orders. A reproducible ETL (`build_clean_data.py`, `data_loaders.py`) normalises line and SKU codes, drops non-productive records (cleaning, line stops), classifies each changeover, and writes tidy tables to `clean_data/` (Table 1).

File	Key columns	Rows	Role in the study
`historical_weeks.csv`	of, fecha, line, sku, marca, hl, oee	2,141	per-order OEE/volume history
`changeovers.csv`	line, prev/next_node, edge_type, hours	978	aggregated transition table
`frames_2025.csv`	week, line, prev/next_node, count	43,390	weekly transition frames
`nodes_2025.csv`	week, line, node, sku, degree	9,574	weekly node degrees
`black_spots_2025.csv`	line, prev/next_node, edge_type, oee	79	flagged anomalous edges
`throughput_rates.csv`	sku, line, rate	251	median HL/h per (SKU, line)
`historical_pairs.csv`	sku, line	253	observed SKU–line eligibility
`demand.csv`	sku, line, hl_total	28	target-week demand

Table 1: Cleaned datasets produced by the ETL and used throughout the report.

Maintenance de-contamination. A changeover record is only informative about *sequencing* if its OEE was not dominated by a maintenance event. Let $r = \text{maintenance time}/H_{\text{tot}}$ for an order. We flag the order as contaminated when $r > 0.20$; the 293/2,141 orders (13.7%) above this threshold are inspected, and the 105 above $r > 0.50$ have their OEE imputed by the SKU–line median. This conservative rule prevents a maintenance stoppage from masquerading as a costly changeover.

Graph representation. The unit of analysis is the **node**, the Cartesian product of four physical attributes; a changeover is a directed **edge** between two nodes, labelled by the *single* attribute that changes (Table 2). This factorisation is what lets us speak of "types" of changeover and, later, order them by difficulty.

Node = Brand × vol × pack × Envase		**Edge type** (attribute that changes)	
Brand	20 commercial brands	C_pack	same brand+vol, different multipack
vol	1/3, 1/2 or 2/5 litre	C_brand	different brand
pack	UNI, P6, P12, P24, RETR	C0_self	same node (lot restart)
Envase	label-reference code KR####	C_envase	same brand+vol+pack, different label

Table 2: Definition of graph nodes and the five changeover (edge) types.

A priori, engineering intuition orders these types by expected effort: a multipack change (C_pack) only re-tools the end-of-line packer and should be cheapest; a brand change (C_brand) swaps the can artwork and recipe and should be moderate; a lot restart (C0_self) carries fixed warm-up overhead; and a label reference change (C_envase), although it sounds trivial, can require a full re-registration of the labeller and quality re-validation. Whether this expected ordering holds in the data — and in particular whether C_envase really is the worst — is exactly hypothesis H3, tested in Section 4.3. Stating the expectation in advance turns the analysis into a genuine confirmatory test rather than a post-hoc rationalisation.

Analysis dataset. For the statistical study (Sections 3–4) we keep every changeover with a recorded duration and a defined predecessor: **1,378 changeovers** spanning **179 nodes** and **1,015 directed edges**. The response variable is the changeover duration in hours; its empirical distribution is strongly right-skewed (Fig. 1), with the mean well above the median, which rules out a Gaussian working model and motivates the positive, skewed families used in Section 3.1. The five edge types are very unevenly represented: C_brand dominates with 975 observations, then C0_self (141),

C_pack (138), C_envase (103) and the rare C_vol (21). The post-mortem module (Section 5) uses a slightly wider reconstruction (1,986 transitions, 188 nodes, 1,358 edges) because it keeps transitions without a numeric duration for centrality analysis.

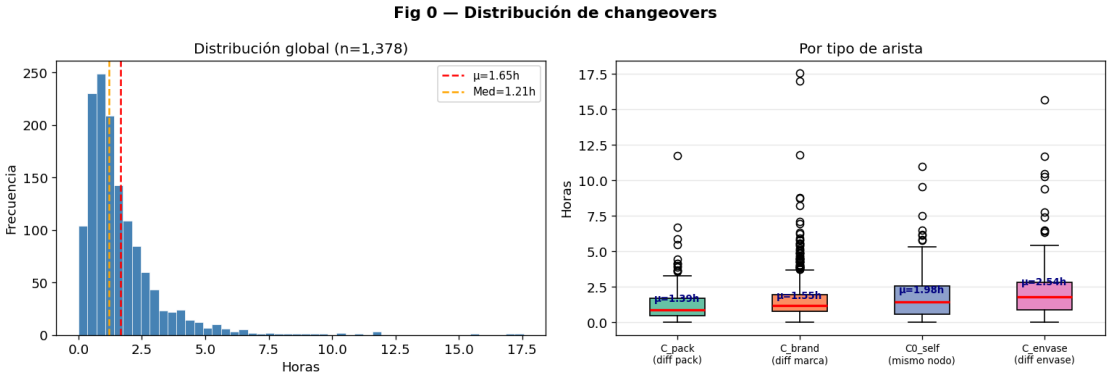


Figure 1: Global distribution of the 1,378 changeover durations (left) and by edge type (right). The right skew and the heavier mean of C_envase versus C_pack are already visible and preview the ordering established in Section 4.3.

2 Problem statement, objectives and hypotheses

The operational problem. Let $\mathcal{S}$ be the set of SKUs to produce in a week, each with a fixed demand V_i (hectolitres), and let $\mathcal{L} = \{14, 17, 19\}$ be the lines. For SKU i on line ℓ the production time is $V_i/\rho_{i\ell}$, where $\rho_{i\ell}$ is the throughput (HL/h); putting j immediately after i on line ℓ costs a changeover time $c_{\ell ij}$. Writing $y_{\ell i} = 1$ if line ℓ produces i and $x_{\ell ij} = 1$ if i immediately precedes j on ℓ, the weekly hours of line ℓ are

$$H_\ell \;=\; s_\ell + \sum_i y_{\ell i}\,\frac{V_i}{\rho_{i\ell}} \;+\; \sum_{i \neq j} x_{\ell ij}\, c_{\ell ij}, \tag{3}$$

with start-up s_ℓ. The planner must keep $H_\ell \le \kappa_\ell$ (capacity $\kappa_{14} = 110$, $\kappa_{17} = \kappa_{19} = 115\,\text{h}$), respect format eligibility, and honour urgent orders. *Minimising $\sum_\ell H_\ell$ is equivalent to maximising the slack $\sum_\ell(\kappa_\ell - H_\ell)$ and, because the only movable term is the changeover sum, to minimising changeover loss — hence to protecting OEE.* The whole pipeline therefore hinges on a credible estimate of $c_{\ell ij}$, which is exactly what the statistical analysis provides.

Objectives.

O1 Identify, per changeover type, the parametric law that best describes its duration (model identification).

O2 Build a defensible, *self-updating* cost per changeover usable as a Bayesian prior for next-year planning, and verify it remains valid out of sample.

O3 Test whether changeover types and lines can be *ordered* by difficulty, so the optimiser can be steered with a few interpretable rules.

O4 Quantify the operational impact (lost hL, recovered slack, urgency feasibility) of acting on these results through the scheduler.

Statistical hypotheses. The two methods translate into four formally testable hypotheses.

H1 *(model identification)* Each edge-type duration follows an identifiable positive-skew family among Gamma, Weibull, LogNormal and Inverse Gaussian. Selection is by AIC; KS and CvM provide diagnostics.

H2 *(stationarity)* The duration distribution is stable between H_1 (Jan–Jun) and H_2 (Jul–Dec): $H_0 : F_{H_1} = F_{H_2}$ vs. $H_A : F_{H_1} \neq F_{H_2}$, tested with two-sample KS, AD and CvM. Non-rejection licenses using H_1 as an informative prior.

H3 *(ordered edge types)* H_0 : the four types share one distribution; $H_A : m_{\mathrm{C\_pack}} \leq m_{\mathrm{C\_brand}} \leq m_{\mathrm{C0\_self}} \leq m_{\mathrm{C\_envase}}$, with at least one strict inequality.

H4 *(ordered lines)* $H_0 : m_{\mathrm{L17}} = m_{\mathrm{L19}} = m_{\mathrm{L14}}$ vs. $H_A : m_{\mathrm{L17}} \leq m_{\mathrm{L19}} \leq m_{\mathrm{L14}}$.

Eligibility and urgencies. Two side constraints shape the feasible region. *Format eligibility* is physical: line 14 runs $\{1/2, 1/3\}$, line 17 only $1/3$, line 19 $\{1/2, 1/3, 2/5\}$, and — a stricter, data-driven rule we adopt — a SKU is admissible on a line only if that exact SKU was produced there in 2025, so the optimiser never invents an untested pairing. *Urgencies* are mandatory orders that must appear early in a line's sequence; we encode them as a hard upper bound on the position variable ($u_{\ell i} \leq 2$). When an urgency cannot be honoured without violating format, eligibility or capacity, the tool returns *infeasible* rather than a fabricated schedule — the only acceptable behaviour for a planning aid that operators must trust.

Information available. Beyond the 2025 history we have the target week (18–22 May 2026): 28 SKUs and 36,933 hL of demand, plus the as-built production log of that week, used in Section 5.6 for a plan-versus-reality reconciliation.

3 Statistical methodology

We use two complementary families of techniques. Method 1 is parametric and Bayesian; Method 2 is non-parametric and permutation-based. Both are deliberately distribution-aware: the strong right skew of the response (Fig. 1) makes normal-theory tools (ANOVA, t-tests) inappropriate.

Why these two, and why together. The choice is dictated by the two questions the business asks. "*How long will this changeover take, and how sure are we?*" is an estimation question with a need for an honest, updatable uncertainty — the natural home of *Bayesian* inference, whose posterior *is* a calibrated probability statement and whose conjugate form lets the plant fold in each new week at negligible cost. "*Is type/line A really worse than B?*" is a testing question where we refuse to assume a distributional shape — the natural home of *permutation* inference, which is exact under exchangeability and immune to the skew that would invalidate a parametric F-test. The two methods are also mutually reinforcing: Method 1 supplies the numeric cost the optimiser consumes, while Method 2 certifies the qualitative ordering that makes that cost interpretable and gives the planner simple rules. Using only one would leave either the magnitudes or the ranking unjustified.

3.1 Method 1 — Goodness-of-fit and Bayesian conjugate inference

3.1.1 Candidate families and why

Changeover durations are positive and right-skewed, so we consider four classical positive laws whose hazard shapes cover the plausible mechanisms (a setup that gets harder, or easier, the longer

it runs):

$$\text{Gamma}(\alpha,\theta):\ f(x)=\tfrac{1}{\Gamma(\alpha)\theta^{\alpha}}x^{\alpha-1}e^{-x/\theta},\qquad \text{Weibull}(k,\lambda):\ f(x)=\tfrac{k}{\lambda}\big(\tfrac{x}{\lambda}\big)^{k-1}e^{-(x/\lambda)^{k}},$$
$$\text{LogNormal}(\mu,\sigma):\ f(x)=\tfrac{1}{x\sigma\sqrt{2\pi}}e^{-\frac{(\ln x-\mu)^2}{2\sigma^2}},\qquad \text{InvGauss}(\mu,\lambda):\ f(x)=\sqrt{\tfrac{\lambda}{2\pi x^3}}\,e^{-\frac{\lambda(x-\mu)^2}{2\mu^2 x}}.$$

Each is fitted on H_1 by maximum likelihood with the location fixed at zero, so every family has $k=2$ free parameters and the comparison is fair.

3.1.2 Model selection: AIC with KS and CvM diagnostics

We rank families by Akaike's Information Criterion [1],

$$\text{AIC}=-2\,\ell(\hat{\theta})+2k, \tag{4}$$

an estimator of the expected Kullback–Leibler divergence to the truth: the first term rewards fit, the second penalises complexity, and the lowest AIC wins. As goodness-of-fit *diagnostics* (not the sole decision rule) we report the one-sample Kolmogorov–Smirnov [9] and Cramér–von Mises [2] statistics,

$$D=\sup_x\big|F_n(x)-F(x)\big|,\qquad W^2=\int\big[F_n(x)-F(x)\big]^2\,dF(x), \tag{5}$$

where F_n is the empirical CDF and F the fitted one; D reacts to the worst local gap, W^2 to the integrated squared gap. With large n both tests reject *any* parametric law, so we lean on AIC and the *size* of D, not the p-value alone — an important caveat exploited in the results.

3.1.3 Bayesian updating: the Gamma–Exponential conjugate model

For planning we want a posterior on the *expected* duration that can absorb each new week. We use the conjugate pair: an Exponential likelihood with rate λ (mean duration $1/\lambda$) and a Gamma prior on λ. With n observations $x_1,\dots,x_n$,

$$p(\lambda)=\text{Gamma}(a_0,b_0)\propto\lambda^{a_0-1}e^{-b_0\lambda},\qquad p(\mathbf{x}\mid\lambda)=\lambda^{n}e^{-\lambda\sum_i x_i}, \tag{6}$$

so by Bayes' theorem the posterior keeps the Gamma form (conjugacy),

$$p(\lambda\mid\mathbf{x})\ \propto\ \lambda^{(a_0+n)-1}\,e^{-(b_0+\sum_i x_i)\lambda}\ =\ \text{Gamma}\Big(a_0+n,\ b_0+\textstyle\sum_i x_i\Big). \tag{7}$$

The quantity of interest is the expected duration $\mathbb{E}[X]=\mathbb{E}[1/\lambda]$. Since $1/\lambda$ is inverse-Gamma, its posterior mean is

$$\mathbb{E}[X\mid\mathbf{x}]=\frac{b_0+\sum_i x_i}{a_0+n-1},\qquad a_0+n>1, \tag{8}$$

and a 95% credible interval for it follows by transforming the Gamma quantiles of λ: if $\lambda_{0.975},\lambda_{0.025}$ are the posterior quantiles, the interval for $\mathbb{E}[X]$ is $[1/\lambda_{0.975},\ 1/\lambda_{0.025}]$. We contrast two priors, updated observation-by-observation along H_2:

$$\textbf{informative:}\quad a_0=n_{H_1},\ b_0=\textstyle\sum_{H_1}x_i\quad(H_1\text{ as a pseudo-sample});$$
$$\textbf{non-informative:}\quad a_0=1,\ b_0=0\quad(p(\lambda)\propto 1).$$

The informative prior injects last semester's scale; the non-informative one learns from H_2 alone. The practical question is whether they agree once enough H_2 data arrive — the Bayesian analogue of asking whether the prior choice matters.

A worked example (C_brand). To make Eqs. (7)–(8) concrete: C_brand has $n_{H_1} = 508$ observations summing to $\sum_{H_1} x_i = 764.9\,\text{h}$, so the informative prior is $\lambda \sim \text{Gamma}(a_0 = 508,\ b_0 = 764.9)$ and its predictive mean is $b_0/(a_0 - 1) = 764.9/507 = 1.509\,\text{h}$. After the first five H_2 changeovers $(1.26, 1.21, 1.32, 2.02, 1.23\,\text{h})$ the posterior is Gamma(513, 771.9) and $\mathbb{E}[X] = 771.9/512 = 1.508\,\text{h}$, with a 95% credible interval of $[1.383, 1.645]\,\text{h}$ — the estimate is essentially unmoved because 508 prior pseudo-observations dwarf five new ones. The non-informative prior instead starts undefined ($a_0 = 1$ gives no mean) and, after the same five points, sits at 1.41 h with a much wider interval $[0.60, 3.20]\,\text{h}$. Both trajectories are shown in Fig. 3; they meet once H_2 is exhausted.

3.1.4 Stationarity tests

Whether the H_1 prior is still valid in H_2 is a two-sample distribution-equality question. We use three complementary statistics: KS (supremum gap), Anderson–Darling [3] (which up-weights the tails, where long changeovers live) and Cramér–von Mises (uniform weight). The two-sample AD statistic for $K = 2$ samples of sizes n_1, n_2 ($N = n_1 + n_2$) is

$$A^2 = \frac{1}{N} \sum_{k=1}^{2} \frac{1}{n_k} \sum_{j=1}^{N-1} \frac{\left(N M_{kj} - n_k j\right)^2}{j\,(N - j)}, \tag{9}$$

with M_{kj} the count of sample-k observations $\leq$ the j-th ordered pooled value. Non-rejection ($p > 0.05$) is evidence of a stationary process. The three statistics are complementary because they weight the cumulative gap differently (Table 3): reporting all three guards against a test that happens to be blind to the kind of departure present in the data.

Test	Statistic	Sensitivity	Used for
Kolmogorov–Smirnov	$\sup_x \lvert F_n - F \rvert$	worst single point, centre	GoF (1-sample), stationarity
Cramér–von Mises	$\int (F_n - F)^2\, dF$	whole curve, uniform weight	GoF (1-sample), stationarity
Anderson–Darling	tail-weighted $\int$	tails (long changeovers)	stationarity (2-sample)

Table 3: The three empirical-distribution-function tests and where each is sensitive. Together they cover central and tail departures.

3.1.5 Empirical-Bayes per-edge priors

Finally, individual edges are sparse: most occur once or twice. Edges with $n \geq 2$ observations in H_1 receive their own Gamma prior; the rest fall back to the global Gamma fit, which acts as a hierarchical regulariser. This is the standard partial-pooling idea [4]: borrow strength from the population where the cell is thin, trust the cell where it is rich.

3.2 Method 2 — Permutation tests for ordered alternatives

Because the response is heavily skewed and several groups are small, H3–H4 are tested with *rank-based permutation* procedures that assume nothing about the shape of the distribution. The logic is exchangeability: *if* H_0 holds, the group labels carry no information, so reshuffling them leaves the joint distribution unchanged; repeating a statistic over $B = 10{,}000$ random relabelings builds its exact (Monte-Carlo) null distribution [5]. The p-value is $(\#\{T^{(b)} \geq T_{\text{obs}}\} + 1)/(B + 1)$.

Omnibus equality. With pooled ranks, mean within-group rank R_i, sizes n_i and grand mean $\bar{R}$, the dispersion statistic

$$Q = \sum_{i=1}^{k} n_i\,(R_i - \bar{R})^2 \tag{10}$$

detects *any* difference among the k groups (it is the Kruskal–Wallis numerator, here calibrated by permutation rather than χ^2).

Ordered alternatives. A significant Q only says "some difference". To test a *direction* we use order-aware statistics. For the $k = 4$ edge types we use the linear rank contrast

$$T = 3R_4 + R_3 - R_2 - 3R_1, \tag{11}$$

whose coefficients $(-3, -1, +1, +3)$ make large T evidence for the increasing order $m_1 \le m_2 \le m_3 \le m_4$. For the $k = 3$ lines we use the Jonckheere–Terpstra statistic [7, 15],

$$JT = \sum_{i<j} U_{ij}, \qquad U_{ij} = \#\{(a, b) : x_{ia} < x_{jb}\}, \tag{12}$$

the canonical distribution-free test against an ordered trend, where U_{ij} is the Mann–Whitney count of how often group j exceeds group i.

Post-hoc. After a global rejection we localise the effect. For edge types we test each ordered pair with $T_{ij} = R_j - R_i$ over the $\binom{4}{2} = 6$ pairs and control the family-wise error with Bonferroni ($\alpha = 0.05/6 = 0.0083$). For lines we use one-sided Mann–Whitney tests [8] with $\alpha = 0.05/3 = 0.0167$.

4 Results of the statistical analysis

The temporal split gives $n_{H_1} = 728$ and $n_{H_2} = 650$ changeovers.

4.1 Exploratory analysis

Before any inference, the shape of the data justifies the modelling choices. The 1,378 durations are positive, right-skewed and heavy-tailed: the median sits below the mean for every group (Tables 7 and 9), the classic signature of a process with a floor (a changeover cannot take zero time) and a long upper tail (occasional very hard changeovers). Skewness is most pronounced for C_envase, whose mean (2.54 h) exceeds its median (1.78 h) by 43%, and mildest for C_pack (1.39 vs 0.87 h). The five edge types are also wildly unbalanced — C_brand alone is 71% of the data and C_vol barely 1.5% — which is itself informative (brand changes are the everyday event; volume changes are rare and operationally segregated) and which is precisely why a rank-based, permutation calibration is preferable to a parametric one that would be dominated by the largest group. These features — positivity, skew, imbalance — are exactly what Method 1's positive families and Method 2's distribution-free tests are designed to handle.

4.2 Method 1 — family selection, priors and stationarity

Table 4 reports the AIC-selected family per edge type. *Gamma is not imposed*: it wins for the two largest groups (C_pack and C_brand) but Inverse Gaussian, Weibull and LogNormal win for C_vol, C0_self and C_envase respectively, each by a small AIC margin over Gamma (e.g. ΔAIC of only +1.9 for C_vol, +0.2 for C0_self, +1.4 for C_envase). The diagnostic p-values are large for every group *except* C_brand, whose $n = 508$ makes even a tiny departure significant — precisely the large-n behaviour of KS/CvM anticipated in Section 3.1; the QQ-plots in Fig. 2 nonetheless show an adequate fit. The implied prior mean duration already previews the ordering of Section 4.3: 1.20 h for C_pack up to 2.49 h for C_envase.

Reading the fitted shapes. The winning families are physically interpretable through their hazard. C0_self (lot restart) is best fit by a Weibull with shape $k \approx 1.06$, i.e. an almost constant hazard — a restart is a memoryless re-warm-up. C_envase (label change) is LogNormal, a multiplicative-error law with a fat upper tail, matching the operational reality that most label swaps are quick but a few cascade into long re-registrations. C_pack and C_brand are Gamma with shape $\alpha \approx 1.2$ (> 1, so an increasing hazard that eventually saturates). These shapes are not mere curve-fitting: they tell the planner *why* C_envase is dangerous — not a higher floor but a heavier tail — which is exactly the kind of changeover SMED programmes target.

Edge type	n_{H_1}	Best family	AIC	KS p	CvM p	μ_{prior} (h)
C_pack	79	Gamma	189.2	0.520	0.413	1.198
C_brand	508	Gamma	1422.7	0.000	0.000	1.506
C_vol	15	InvGauss	45.9	0.909	0.914	1.689
C0_self	71	Weibull	221.5	0.723	0.890	1.704
C_envase	55	LogNormal	202.2	0.879	0.903	2.485
GLOBAL	728	Gamma	2096.5	0.000	0.001	1.565

Table 4: Goodness-of-fit on H_1: family with minimum AIC per edge type, with KS and CvM diagnostics and the prior mean duration. The winning family is data-driven; Gamma dominates only the two high-volume types.

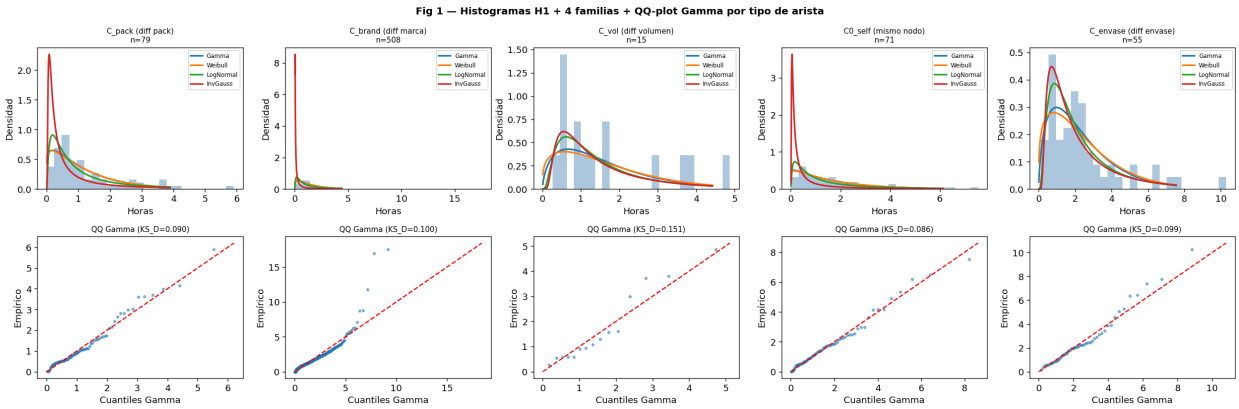


Figure 2: Per-type histograms on H_1 with the four fitted densities (top) and Gamma QQ-plots (bottom). Points on the diagonal indicate an adequate fit; the heavier upper tail of C_envase is apparent.

Posterior updating. Figure 3 shows the conjugate update of Eq. (7) for two contrasting types. The informative posterior starts at the H_1 mean and barely moves — its 95% interval is already tight — whereas the non-informative posterior starts from the first H_2 point and climbs towards it. After consuming all of H_2 the two agree to within a fraction of an hour (Table 5): the largest gap is -0.26 h (-12%) for the medium-sized C0_self and the smallest -0.05 h (-3%) for the data-rich C_brand. In other words, *with enough new data the prior is washed out*, exactly as Bayesian asymptotics predict; the informative prior simply buys faster convergence and narrower intervals for the sparse types — which is precisely where planning needs help. Figure 4 contrasts the full posterior densities before and after seeing H_2.

Edge type	n_{H_1}	n_{H_2}	Posterior (inf.)	Posterior (unif.)	Δ (h)	Δ (%)
C_pack	79	59	1.399	1.644	−0.245	−14.91
C_brand	508	467	1.552	1.598	−0.047	−2.92
C_vol	15	6	1.772	1.684	+0.088	+5.20
C0_self	71	70	1.995	2.259	−0.264	−11.71
C_envase	55	48	2.561	2.669	−0.108	−4.04

Table 5: Final posterior expected duration after consuming all of H_2: informative vs. non-informative prior. The two agree closely once data accumulate.

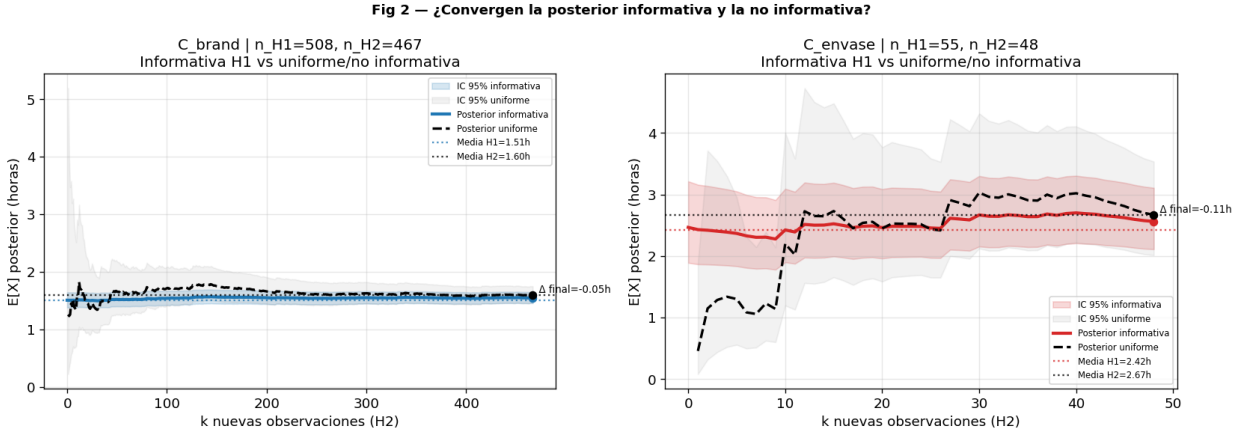


Figure 3: Sequential Gamma–Exponential posterior mean (with 95% band) along H_2 for C_brand and C_envase: informative prior (colour) vs. non-informative (black). They converge as observations accrue.

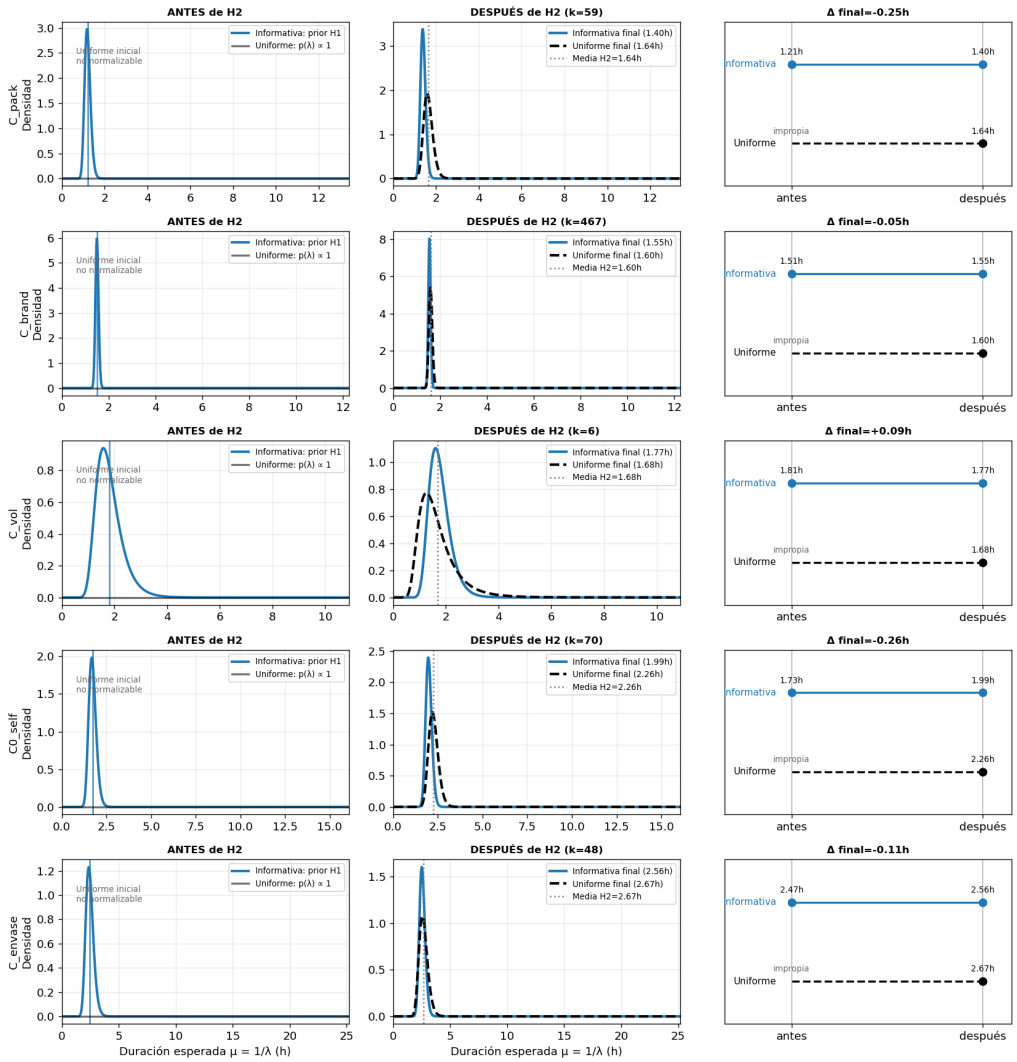
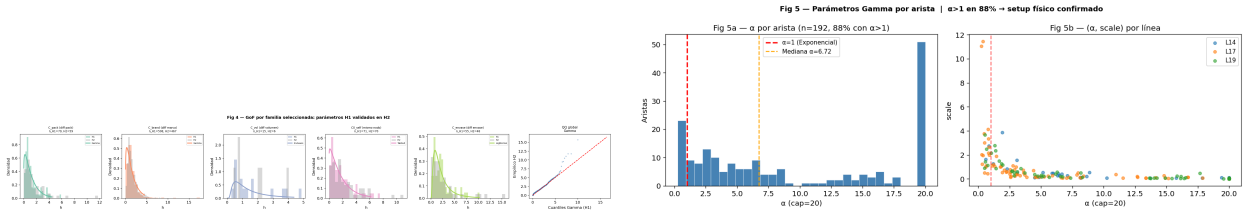


Figure 4: Prior-to-posterior densities of the expected duration before and after H_2, under both priors. The data dominate: the two posteriors concentrate on almost the same value.

Stationarity (H2). Table 6 gives the two-sample tests. For four of the five types ($p > 0.05$ on all three statistics) we *cannot reject* equality of the H_1 and H_2 distributions: the process is stationary and the H_1-based prior is valid out of sample. The exception is C_brand (and, by aggregation, the global pool), where the very large sample exposes a small but detectable shift — a *precision* effect, not a managerially large change, consistent with the -2.9% posterior gap in Table 5. A direct $H_1 \to H_2$ re-validation of each selected family (Fig. 5a) confirms the same picture. The empirical-Bayes layer then assigns a specific Gamma prior to 192 of the 1,015 edges (18.9%); the remaining 823 (81.1%) fall back to the global Gamma($\alpha = 1.208, \theta = 1.295$), $\mu = 1.56$ h (Fig. 5b).

Edge type	n_{H_1}	n_{H_2}	KS D	KS p	AD stat	CvM p
C_pack	79	59	0.158	0.325	0.095	0.301
C_brand	508	467	0.089	0.039	5.170	0.006
C_vol	15	6	0.233	0.938	−1.148	0.954
C0_self	71	70	0.174	0.211	0.644	0.158
C_envase	55	48	0.154	0.516	−0.086	0.535
GLOBAL	728	650	0.093	0.005	5.856	0.002

Table 6: Stationarity (H_1 vs H_2): two-sample KS, Anderson–Darling and Cramér–von Mises. $p > 0.05$ (non-rejection) for every type except the high-volume C_brand.



(a) $H_1 \to H_2$ validation of the selected families. **(b)** Per-edge empirical-Bayes Gamma priors.

Figure 5: Out-of-sample fit and hierarchical priors. Left: the family chosen on H_1 still describes H_2 for the stationary types. Right: distribution of the per-edge shape parameters; most edges fall back to the global prior.

4.3 Method 2 — ordered differences between types and lines

Edge types (H3). On the full sample the four edge types have clearly increasing mean ranks (Table 7). The omnibus permutation test overwhelmingly rejects equality ($Q = 4.67 \times 10^6$, $p = 10^{-4}$), and the ordered contrast confirms the hypothesised direction ($T = 863.1$, $p = 10^{-4}$; Fig. 6). The Bonferroni post-hoc (Table 8) shows the ordering is driven by the *extremes*: every contrast involving C_pack (cheapest) or C_envase (most expensive) is significant, whereas the adjacent middle pairs C_brand ≤ C0_self and C0_self ≤ C_envase are not — brand changes and lot restarts sit on a statistical plateau. The headline result is that **C_envase ($\mu = 2.54$ h) is significantly more expensive than C_brand ($\mu = 1.55$ h)**: swapping a label reference costs more than switching brand entirely, an unintuitive finding with direct sequencing consequences.

Edge	n	Mean	Median	R_i
C_pack	138	1.389	0.870	555.1
C_brand	975	1.550	1.208	674.1
C0_self	141	1.981	1.435	727.8
C_envase	103	2.537	1.784	824.9

Table 7: Duration statistics and mean ranks by edge type.

Ordered pair	Δmean	p_{Bonf}	Sig.
C_pack ≤ C_brand	0.161	0.0030	yes
C_pack ≤ C0_self	0.592	0.0066	yes
C_pack ≤ C_envase	1.148	0.0006	yes
C_brand ≤ C0_self	0.431	0.352	no
C_brand ≤ C_envase	0.987	0.0012	yes
C0_self ≤ C_envase	0.556	0.231	no

Table 8: Ordered post-hoc (permutation, Bonferroni $\alpha = 0.0083$).

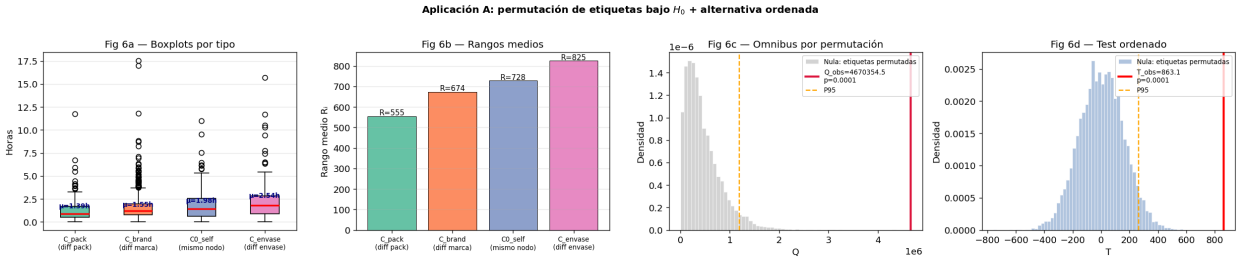


Figure 6: Application A (edge types). From left: boxplots, mean ranks, omnibus null distribution of Q, and null distribution of the ordered statistic T. The observed statistics lie far in the right tail of their permutation nulls.

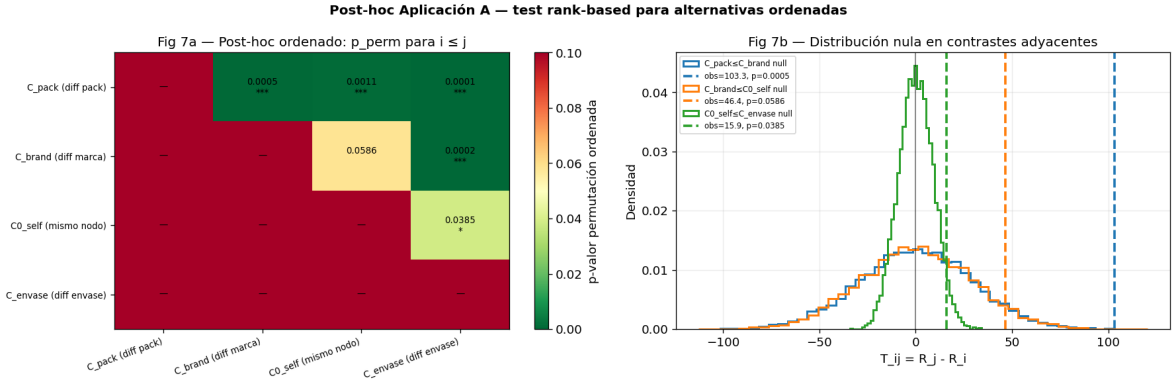


Figure 7: Ordered post-hoc for edge types. Left: matrix of one-sided permutation p-values for each ordered pair $i \leq j$ (green = significant after Bonferroni). Right: null distributions of the adjacent contrasts that define the proposed chain. The significant cells trace the path C_pack → C_brand → C_envase.

Lines (H4). The Jonckheere–Terpstra test rejects equality of the three lines ($JT = 324{,}709$, $p = 3 \times 10^{-4}$; Fig. 8). The post-hoc (Tables 9–10) localises the effect: L14 is significantly slower than both L17 and L19 ($p < 10^{-4}$), while L17 and L19 are statistically indistinguishable ($p = 0.195$). Line 14 is therefore the bottleneck for long changeovers and should be loaded with sequences that minimise expensive (label-switching) transitions.

Line	n	Mean	Median	R_i
L17	609	1.580	1.150	659.1
L19	547	1.560	1.179	677.7
L14	222	2.087	1.585	801.9

Table 9: Duration statistics and mean ranks by line.

Contrast	p (one-sided)	Sig.
L17 < L19	0.195	ns
L17 < L14	$< 10^{-5}$	***
L19 < L14	3×10^{-5}	***

Table 10: Mann–Whitney post-hoc (Bonferroni $\alpha = 0.0167$).

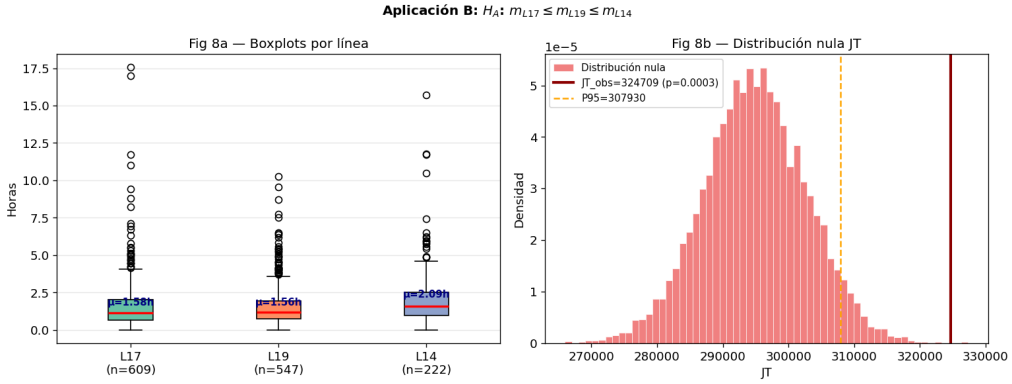


Figure 8: Application B (lines): boxplots by line and the Jonckheere–Terpstra null distribution with the observed JT far in the right tail.

Synthesis. Table 11 collects the four decisions. Method 1 delivers a validated, self-updating cost per changeover; Method 2 certifies how to order those costs. Both point the same way and feed directly into the optimiser of Section 5.

Method	H_0 / H_A	Statistic	Decision
GoF (global, validated on H_2)	selected family vs. alt.	$\mathrm{AIC}_{H_2} = 2022.4$	Gamma retained
Stationarity (per type)	$F_{H_1} = F_{H_2}$	KS/AD/CvM	not rejected (4/5 types)
Omnibus permutation ($k = 4$)	all labels exchangeable	$Q = 4.67 \times 10^6$	reject H_0 ($p = 10^{-4}$)
Ordered contrast ($k = 4$)	C_pack$\le \cdots \le$C_envase	$T = 863.1$	reject H_0 ($p = 10^{-4}$)
Jonckheere–Terpstra ($k = 3$)	L17≤L19≤L14	$JT = 324{,}709$	reject H_0 ($p = 3\times10^{-4}$)

Table 11: Summary of the four hypothesis tests and their decisions.

5 Optimization: methodology and results

The statistical model is only useful if it changes a decision. This section turns the validated changeover cost into a weekly schedule. We first build the cost itself from a directed-graph post-mortem, then state three interchangeable optimisation engines, and finally report their results and a confrontation with reality.

5.1 From history to cost: the directed-graph post-mortem

For optimisation we need a number on every arc $i \to j$. We treat each historical transition as a sample of OEE *degradation*,

$$\delta_{ij} = \overline{\mathrm{OEE}}_{\mathrm{expected}} - \mathrm{OEE}_{\mathrm{observed}}(i \to j), \tag{13}$$

and flag an arc as a **black spot** when it is both recurrent (count ≥ 2) and an outlier in degradation,

$$\delta_{ij} > \mu_\delta + 1.5\,\sigma_\delta, \tag{14}$$

a robust 1.5σ rule on the line's degradation distribution. The avoidable loss of an arc is estimated as

$$\widehat{\mathrm{hL\ lost}}_{ij} = \delta_{ij} \times \mathrm{hL}_{ij}, \tag{15}$$

the degradation scaled by the volume that flowed through it. The same transition matrix is read as a directed graph $G_\ell = (N_\ell, E_\ell)$ per line, on which we compute weighted in/out-degree, betweenness and PageRank to rank "hard-to-reach" destination nodes. Graph densities are low (0.060, 0.065,

0.041 for L14/L17/L19), i.e. each product only ever follows a handful of others — a structure the optimiser exploits to find cheap neighbours. The arc cost that enters the optimiser is either this OEE degradation (CP-SAT objective) or its translation into changeover *hours* via Eq. (3) (graph/GA objective); at fixed volume the two are monotone, since worse OEE means more real time for the same output.

5.2 Estimating the model parameters $\rho_{i\ell}$ and $c^h_{\ell ij}$

Equation (3) needs two estimated quantities per line. The **throughput** $\rho_{i\ell}$ is the *median* HL/h of SKU i on line ℓ across its 2025 orders,

$$\hat{\rho}_{i\ell} = \text{median}\{\text{HL/h} : (\text{SKU} = i, \text{line} = \ell)\}, \tag{16}$$

the median being far more robust than the mean to the occasional bad shift; when a pair (i, ℓ) has no history we back off to the SKU's plant-wide median and then to the plant mean, with a hard floor of 20 HL/h. The estimated medians are 82.4, 144.2 and 140.5 HL/h for L14, L17 and L19 — L14 is intrinsically the slowest line, compounding the changeover penalty established in Section 4.3. The **changeover hours** $c^h_{\ell ij}$ are the posterior summaries of the Gamma-conjugate model (Section 3.1), looked up on the arc $\text{node}(i) \to \text{node}(j)$; a missing arc falls back to the line-mean changeover, inflated by a factor 2.5 when the formats differ, and the result is clipped to $[0, 12]$ h to bound simulator pathologies.

5.3 Engine 1 — exact CP-SAT integer program

The exact model is a capacitated assignment-plus-sequencing program solved with Google OR-Tools CP-SAT [13]. With binaries $y_{\ell i}$ (assignment), $x_{\ell ij}$ (immediate succession) and integer position variables $u_{\ell i}$, and arc cost $c_{\ell ij}$ from Section 5.1,

$$\min \sum_{\ell,i,j} c_{\ell ij}\, x_{\ell ij} \tag{17}$$

$$\text{s.t.} \quad \sum_{\ell} y_{\ell i} = 1 \qquad \forall i \qquad \text{(each SKU produced once)} \tag{18}$$

$$\sum_{j} x_{\ell ij} = y_{\ell i}, \quad \sum_{j} x_{\ell ji} = y_{\ell i} \qquad \forall \ell, i \qquad \text{(flow conservation, virtual start/end)} \tag{19}$$

$$u_{\ell i} - u_{\ell j} + |\mathcal{S}|\, x_{\ell ij} \le |\mathcal{S}| - 1 \qquad \forall \ell, i \neq j \qquad \text{(MTZ sub-tour elimination)} \tag{20}$$

$$s_\ell + \sum_i y_{\ell i} \tfrac{V_i}{\rho_{i\ell}} + \sum_{i \neq j} c^h_{\ell ij} x_{\ell ij} \le \kappa_\ell \qquad \forall \ell \qquad \text{(weekly capacity)} \tag{21}$$

$$y_{\ell i} = 0 \text{ if format}(i) \notin \mathcal{F}_\ell; \quad u_{\ell i} \le 2 \text{ if } i \text{ urgent} \qquad \text{(eligibility, urgencies)} \tag{22}$$

The Miller–Tucker–Zemlin constraints [11] forbid disconnected cycles so that each line's selected arcs form a single Hamiltonian path; the position variables double as the lever to force urgent SKUs early. The solver runs with a 60 s budget and a feasible heuristic warm-start (used only to seed the search, not to bias the optimum). This engine optimises OEE degradation directly through Eq. (17).

5.4 Engine 2 — behaviour-graph hours model

The second engine reformulates the objective as *hours* rather than degradation, which is what the plant ultimately budgets. It learns one directed behaviour graph per line from 2025, with arc weight equal to the expected transition hours, and applies a strict eligibility rule: a SKU is admissible on a

line only if (i) its physical format is allowed ($\mathcal{F}_{14} = \{1/2, 1/3\}$, $\mathcal{F}_{17} = \{1/3\}$, $\mathcal{F}_{19} = \{1/2, 1/3, 2/5\}$) *and* (ii) that exact SKU was produced on that line in 2025. A SKU with no historical precedent is reported as *non-modellable* rather than being forced into a fabricated recommendation — a deliberately conservative stance. The objective minimises $\sum_\ell H_\ell$ from Eq. (3), and the reported quantity is the *slack* $\kappa_\ell - H_\ell$, the hours the plan leaves free.

5.5 Engine 3 — genetic algorithm with a Bayesian changeover cost

The third engine is a genetic algorithm (GA) [6] that scales to the full assignment-and-ordering search while keeping every candidate feasible.

Encoding and fitness. A chromosome is a dict-of-lists $\{14 : [\dots], 17 : [\dots], 19 : [\dots]\}$ in which every weekly SKU appears exactly once; the list order is the production sequence. The fitness is the simulated weekly hours plus penalties,

$$\text{fit} = \sum_\ell H_\ell + w_{\text{inc}} \sum_{\text{infeasible}} 1 + \sum_\ell \Big[w_{\text{inc}}(1 + o_\ell^2) + w_{\text{cap}}\big(e^{o_\ell/5} - 1\big) \Big] \mathbf{1}\{o_\ell > 0\} + w_{\text{urg}} \frac{(\text{pos} - \text{cut})_+}{|\text{seq}|}, \quad (23)$$

with overshoot $o_\ell = (H_\ell - \kappa_\ell)$ and weights $w_{\text{inc}} = 10^4$, $w_{\text{cap}} = 100$, $w_{\text{urg}} = 500$. The capacity term is convex (quadratic + exponential) so the search is gently pushed back under the hard cap rather than discontinuously rejected.

Bayesian arc cost (link to Method 1). The changeover hours $c^h_{\ell ij}$ used inside the simulator are *not* raw means: they are posterior summaries of the same Gamma-conjugate model of Section 3.1. For each arc the line carries a prior $\lambda \sim \text{Gamma}(a_0, b_0)$ and the observed samples update it to $\text{Gamma}(a_0 + n,\, b_0 + \sum x)$, exactly Eq. (7); the cost can be read off as the posterior-predictive mean $b/(a-1)$ (Eq. (8)) or as a conservative upper edge of the $1-\epsilon$ highest-density interval of the posterior duration. The statistical model is thus not a side-analysis: it *is* the optimiser's cost oracle.

Operators and parameters. Generation 0 is built by a smart initialiser that places every SKU on an eligible line, so no individual is born infeasible. Selection is by tournament ($k = 3$); crossover inherits half of one parent's line assignments and fills the rest from the other parent, then order-crosses within each line; mutation is dual — intra-line *swap* to improve a local neighbour cost, and inter-line *migrate* to off-load the most overloaded line (typically L19). With elitism = 4, population = 60 and 150 generations, the GA converges in a few seconds.

Convergence and penalty calibration. Because generation 0 is feasible and the capacity penalty in Eq. (23) is smooth, the search descends quickly: the best individual typically reaches the baseline hours within the first few dozen generations and the population median follows, with no infeasible elite surviving. The penalty weights are deliberately separated by orders of magnitude — $w_{\text{inc}} = 10^4$ (a structural violation must never be traded for a few saved hours), $w_{\text{cap}} = 100$ (capacity is hard but the convex term gives a usable gradient), $w_{\text{urg}} = 500$ (urgencies bend but do not break the schedule) — so the lexicographic intent (feasibility $\gg$ capacity $\gg$ urgency $\gg$ hours) is encoded directly in the scalar fitness. This is why the stochastic GA lands on exactly the CP-SAT optimum rather than near it.

Why three engines. The three engines are not redundant: they are a cross-check (Table 12). CP-SAT gives a *provably* optimal degradation for the modelled instance but scales poorly; the graph model is transparent and fast but only reorders within a fixed objective; the GA scales to the full assignment-and-ordering search and carries the Bayesian cost natively. Agreement between an exact

solver and a stochastic search is the strongest available evidence that the reported plan is genuinely optimal rather than an artefact of one method.

	CP-SAT	**Behaviour graph**	**Genetic algorithm**
Objective	OEE degradation (Eq. 17)	weekly hours (Eq. 3)	weekly hours + penalties
Decision	assign + sequence	assign + sequence	assign + sequence
Cost source	post-mortem matrix	graph arc hours	Bayesian posterior
Guarantee	exact (≤ 60 s)	exact on subset	heuristic, near-optimal
Scales to full week	limited	yes	yes

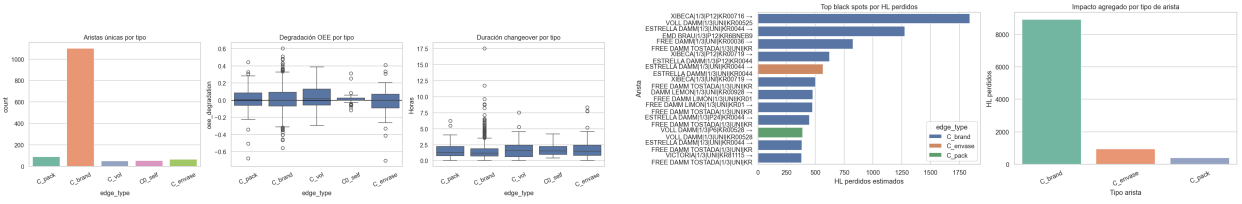
Table 12: The three optimisation engines and their trade-offs. Their agreement on the same plan validates both the cost model and the result.

5.6 Optimization results

Black spots. The post-mortem isolates **21 black spots**. The worst, `XIBECA|1/3|P12` → `VOLL DAMM|1/3|UNI` on L17, degrades OEE by 0.356 and is estimated at 1,840 hL. Aggregated by line (Table 13) the black spots account for an estimated **10,251 hL** of lost output over the year, heavily concentrated on L17. The graph centrality view (Fig. 9) ranks the "hardest-to-reach" destinations as `FREE DAMM TOSTADA|1/3|UNI` (accumulated degradation 3.34) and `ESTRELLA DAMM|1/3|UNI|KR00443` (3.00) — both *label-reference-heavy* nodes, independently corroborating the C_envase signal from the permutation test.

Line	Mean OEE 2025	# black spots	Est. hL lost	Worst transition
L14	0.446	2	1,666	ESTRELLA DAMM → EMD BRAU
L17	0.556	12	6,337	XIBECA → VOLL DAMM
L19	0.500	7	2,248	XIBECA → ESTRELLA DAMM (P12)
Total	—	**21**	**10,251**	—

Table 13: Post-mortem summary by line: mean OEE, black-spot count and estimated lost output.



(a) Directed changeover graph (per line). **(b)** Black-spot ranking by estimated hL lost.

Figure 9: Post-mortem of the 2025 history: the directed transition graph and the ranked black spots that drive avoidable loss.

Weekly schedule. For the target week (28 SKUs, 36,933 hL) all three engines converge to the *same* feasible plan: a total of **267.05** h leaving **72.95** h of weekly slack (Table 14). That the heuristic GA and the exact CP-SAT model agree validates both the cost model and the planner's manual baseline: the schedule is already near-optimal. The value of the analysis is therefore not a dramatic re-shuffle but (i) the *quantified* 73 *h of head-room* for urgencies and demand swings, and (ii) the interpretable rule of where to place expensive transitions (keep C_envase off L14).

Line	SKUs	Prod (h)	Changeover (h)	Total (h)	Capacity (h)	Slack (h)
L14	3	68.62	2.00	70.62	110	39.38
L17	12	83.96	11.00	94.96	115	20.04
L19	13	89.47	12.00	101.47	115	13.53
Total	28	241.95	25.00	**267.05**	340	**72.95**

Table 14: Optimised weekly schedule (identical across CP-SAT, graph and GA). All lines feasible; 72.95 h of slack recovered against the contractual capacity.

Plan versus reality. Reconciling the plan with the as-built log of 18–22 May 2026 gives an overall attainment of **88.2**% (38,495 hL produced of 43,659 hL planned; Table 16). The shortfall is uneven: L17 reached only 74.4% — two SKUs (`SK13LN`, `VO13LTNN`) were never produced — while L19 overran to 105.9%, absorbing unplanned urgent SKUs. Re-optimising on the *realised* demand reproduces this stress exactly: L19 is the only infeasible line (117.67 h vs. 115 h, −2.67 h), confirming that L19 acts as the plant's buffer and is the binding constraint when urgencies arrive. When an urgent order is added as a hard constraint (SKU must occupy an early position) the optimiser either schedules it or, if it conflicts with format/eligibility/capacity, *reports infeasibility* rather than fabricating a solution — the honest behaviour an operations tool needs. Table 15 shows the re-optimisation on realised demand: L14 and L17 keep comfortable slack (+46.5 and +19.4 h) while L19 alone breaches capacity, the quantitative signature of the buffer role.

Line	Prod (h)	Changeover (h)	Total (h)	Slack (h)	Feasible?
L14	61.51	2.00	63.51	+46.49	yes
L17	85.62	10.00	95.62	+19.38	yes
L19	106.67	11.00	117.67	−2.67	**no (over cap)**

Table 15: Re-optimisation on the *realised* demand of 18–22 May 2026. Only L19 exceeds its 115 h cap, confirming it as the binding constraint when unplanned urgencies land.

Line	hL plan	hL real	ΔhL	Attainment	Real OEE
L14	8,859.6	7,420.3	−1,439.2	83.8%	0.59
L17	18,297.4	13,605.9	−4,691.5	74.4%	0.54
L19	16,501.9	17,469.2	+967.3	105.9%	0.64
Total	**43,658.9**	**38,495.4**	−5,163.5	**88.2%**	—

Table 16: Plan vs. realised production (week 18–22 May 2026). L17 under-delivers, L19 over-delivers as the urgency buffer.

6 Findings, interpretation and conclusions

What the two methods jointly establish. Method 1 gives a *trustworthy, self-updating cost per changeover*: a per-type parametric law (not forced to Gamma), a conjugate prior whose informative and non-informative versions agree once data accrue (Table 5), and a stationarity check that licenses re-using the 2025 prior in 2026 (Table 6). Method 2 turns that cost into *actionable structure*: a statistically certified ordering of changeover types and of lines (Table 11). The two agree on the central, counter-intuitive message — **a label-reference change (C_envase) is the single most expensive transition, costlier than a full brand change** — and the directed-graph post-mortem corroborates it through the centrality of label-heavy destination nodes.

Operational consequences.

1. *Sequencing rule:* group SKUs that share a label reference so C_envase transitions are rare; prefer C_pack and C_brand boundaries.
2. *Line loading:* keep long, label-switching sequences off the slow L14, and treat L19 as the urgency buffer it empirically is.
3. *Targeted action:* the 21 black spots ($\approx$ 10,251 hL) are a short, ranked worklist for SMED-style changeover reduction [14].
4. *Planning value:* the optimiser confirms $\approx$ 73 h of weekly slack — the quantitative room to absorb demand swings without overtime — and the plan-vs-real gap (88% attainment, L19 overshoot) pinpoints where that room disappears.

From evidence to action. Table 17 closes the loop by mapping each statistical result to the concrete decision it licenses — the deliverable a planner actually uses.

Statistical evidence	Section	Operational action
C_envase is the costliest type ($p < 0.01$)	4.3	batch SKUs by label reference; minimise C_envase
C_brand $\approx$ C0_self (plateau)	4.3	treat brand changes and lot restarts as equal-cost
L14 slowest ($p < 10^{-4}$)	4.3	keep label-switching sequences off L14
Process stationary (4/5 types)	4	reuse 2025 prior for 2026 planning
Informative $\approx$ uninformative posterior	4	cost is data-driven, not prior-driven; trust it
21 black spots, $\approx$ 10,251 hL	5.6	ranked SMED worklist
72.95 h weekly slack	5.6	head-room budget for urgencies
L19 binding under real demand	5.6	route urgencies away from L19 when possible

Table 17: Traceability from statistical evidence to operational decisions.

Quantified impact. The results convert into concrete numbers. The 21 black spots represent an estimated 10,251 hL of annual output lost to avoidable degradation; even a partial SMED attack on the dozen worst transitions on L17 (the line carrying 6,337 hL of that loss) targets the single largest recoverable pool. On the forward-planning side, the 72.95 h of certified weekly slack is the buffer that keeps the plant feasible under the demand swings the plan-vs-real analysis exposed: when realised demand pushed L19 to 117.67 h the slack on L14 and L17 (46.5 and 19.4 h) is exactly the capacity a re-balancing move would draw on. The statistical layer thus pays for itself twice — once by ranking where to cut loss, once by quantifying the head-room that absorbs volatility.

Limitations. KS/CvM reject the parametric and stationarity nulls for C_brand purely because $n = 508$ makes them over-powered; we therefore lean on AIC and effect size, not the p-value alone. The hL impact of black spots is an estimate (degradation $\times$ volume), and the case$\rightarrow$hL conversion has known noise for three SKUs with non-inferable format. Maintenance-contaminated OEE was median-imputed, which is conservative but smooths genuine variance. C_vol ($n = 21$ overall, 15 in H_1) is too sparse for strong claims and is excluded from the ordered contrast. The optimiser's eligibility is intentionally conservative (2025 precedent only), which guarantees realism at the cost of never proposing a genuinely new SKU–line pairing.

Future work. Three extensions follow naturally. First, the per-edge empirical-Bayes priors could be replaced by a fully *hierarchical* model in which edge-, type- and line-level effects are estimated jointly, sharpening the sparse arcs without the hard $n \geq 2$ cut-off. Second, the conjugate cost could be pushed into the CP-SAT objective as a risk-adjusted quantile (e.g. the posterior 90% duration) so the exact optimiser becomes robust, not just expected-value optimal. Third, the stationarity test could be run online as a change-point monitor, raising an alert when a line's changeover distribution drifts — closing the loop from descriptive post-mortem to live process control.

Reproducibility. Every figure and number in this report is regenerated from the raw spreadsheets by the executed notebook `07_statistical_analysis.ipynb` and the modules `post_mortem.py`, `ga_optimizer.py` and `scheduling_cp_sat.py`; the statistical routines are reproduced verbatim in Appendix A. Fixed random seeds make the 10,000-permutation nulls and the GA run deterministic, so the reported p-values and the 267.05 h plan reproduce exactly.

Conclusion. A disciplined statistical treatment of an industrial log — AIC-based family selection, Gamma–Exponential conjugate updating with an explicit stationarity test, and distribution-free permutation tests for ordered alternatives — produced a validated, interpretable cost structure that both explains historical losses and drives a feasible production schedule. Crucially, the statistical model is not decorative: its conjugate posterior is the optimiser's cost oracle, and the ordering it certifies is the rule the optimiser follows. The framework is reproducible and self-improving: each new production week is one more observation for the posterior of Eq. (7), so the cost model — and the schedules it produces — sharpen automatically as DAMM keeps operating.

References

[1] Hirotugu Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974.

[2] T. W. Anderson. On the distribution of the two-sample Cramér–von Mises criterion. volume 33, pages 1148–1159. 1962.

[3] T. W. Anderson and D. A. Darling. Asymptotic theory of certain "goodness of fit" criteria based on stochastic processes. *The Annals of Mathematical Statistics*, 23(2):193–212, 1952.

[4] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. CRC Press, 3rd edition, 2013.

[5] Phillip I. Good. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*. Springer, 3rd edition, 2005.

[6] John H. Holland. *Adaptation in Natural and Artificial Systems*. MIT Press, 1992.

[7] A. R. Jonckheere. A distribution-free k-sample test against ordered alternatives. *Biometrika*, 41(1/2):133–145, 1954.

[8] H. B. Mann and D. R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[9] Frank J. Massey Jr. The Kolmogorov–Smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253):68–78, 1951.

[10] Wes McKinney. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[11] C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326–329, 1960.

[12] Seiichi Nakajima. *Introduction to TPM: Total Productive Maintenance*. Productivity Press, 1988.

[13] Laurent Perron and Vincent Furnon. OR-Tools CP-SAT solver. Google, 2023. https://developers.google.com/optimization.

[14] Shigeo Shingo. *A Revolution in Manufacturing: The SMED System*. Productivity Press, 1985.

[15] T. J. Terpstra. The asymptotic normality and consistency of Kendall's test against trend. *Indagationes Mathematicae*, 14:327–333, 1952.

[16] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

A Appendix — Reproducible analysis code

The statistical analysis was implemented in **Python** (the language in which the project was developed and executed). For full transparency, Table 18 maps every statistical procedure to its exact Python package and function; the listings that follow contain the corresponding code. All code, data and the executed notebook (`notebooks/07_statistical_analysis.ipynb`) are reproducible end to end with `numpy`, `pandas` [10] and `scipy` [16] (SciPy 1.16, pandas 2.3).

Procedure	Python function	Notes
Method 1 — goodness of fit & Bayesian inference		
MLE fit of families	`scipy.stats.{gamma,weibull_min, lognorm,invgauss}.fit`	location fixed: `.fit(s, floc=0)` ($k = 2$ free parameters)
AIC	`-2*dist.logpdf(s,*p).sum()+2k`	Eq. (4)
Kolmogorov–Smirnov (1-sample)	`scipy.stats.kstest`	GoF vs. fitted CDF
Cramér–von Mises (1-sample)	`scipy.stats.cramervonmises`	uniform-weight GoF
KS (2-sample, H_1 vs H_2)	`scipy.stats.ks_2samp`	stationarity
Anderson–Darling (k-sample)	`scipy.stats.anderson_ksamp`	tail-weighted
Cramér–von Mises (2-sample)	`scipy.stats.cramervonmises_2samp`	stationarity
Conjugate update + 95% CI	custom `sequential_bayes`	`gamma.ppf` quantiles
Method 2 — permutation tests		
Joint ranks	`scipy.stats.rankdata`	mean within-group ranks
Label-permutation null	`numpy.random.Generator.permutation`	10,000 resamples
Line post-hoc	`scipy.stats.mannwhitneyu`	one-sided, Bonferroni
Pair enumeration	`itertools.combinations`	$\binom{4}{2}$ ordered pairs

Table 18: Software-and-methods map: each statistical test and the Python function that implements it. The KS, AD and CvM tests and the Bayesian conjugate update all rely on `scipy.stats`.

A.1 Data preparation, graph construction and goodness of fit

```
"""
Appendix A.1 -- Data preparation, graph construction and Goodness-of-Fit.
Source: notebooks/07_statistical_analysis.ipynb (Sections 1 and 2.1-2.2).

Software stack: pandas (data handling), numpy (numerics),
scipy.stats (parametric families + GoF tests).
"""
import warnings; warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
from scipy import stats
from pathlib import Path

np.random.seed(42)
BASE = Path.cwd() if (Path.cwd() / 'raw_data').exists() else Path.cwd().parent

# --- Load raw changeover log and attach the production line ------------------
xl = pd.read_excel(BASE / 'raw_data/Cambios 14_17_19_ 2025.xlsx')
hw = pd.read_csv(BASE / 'clean_data/historical_weeks.csv')
xl = xl.merge(hw[['of', 'line']].drop_duplicates(),
              left_on='OF', right_on='of', how='left').dropna(subset=['line'])
xl['line'] = xl['line'].astype(int)

```

```
# --- Build the graph node = Marca x vol x pack x Envase ----------------------
def parse_vol(te):
    for v in ['1/3', '1/2', '2/5']:
        if v in str(te):
            return v
    return 'UK'

def parse_pack(mp):
    mp = str(mp).upper()
    if any(k in mp for k in ['PACK 24', 'BANDEJA24', 'B24']): return 'P24'
    if any(k in mp for k in ['PACK 12', 'P12']):              return 'P12'
    if any(k in mp for k in ['PACK C. 6', 'PACK 6']):         return 'P6'
    if any(k in mp for k in ['RETRACTIL', 'RETR']):           return 'RETR'
    if 'PACK' in mp:                                          return 'PACK'
    return 'UNI'

xl = xl.sort_values(['line', 'Fecha Fin']).reset_index(drop=True)
xl['Marca'] = xl['Marca'].str.strip()
xl['vol']   = xl['Tipo Envase'].apply(parse_vol)
xl['pack']  = xl['Material Precio'].apply(parse_pack)
xl['node']  = xl['Marca'] + '|' + xl['vol'] + '|' + xl['pack'] + '|' + xl['Envase']
for col in ['Marca', 'vol', 'pack', 'Envase', 'node']:
    xl[f'prev_{col}'] = xl.groupby('line')[col].shift(1)

ch = xl[xl['Frecuencia Total'].notna() & xl['prev_node'].notna()].copy()
ch = ch.rename(columns={'Frecuencia Total': 'hours', 'Fecha Fin': 'fecha'})
ch['fecha'] = pd.to_datetime(ch['fecha'])

# --- Classify the edge type by the attribute that changes --------------------
def classify_edge(r):
    if r['Marca']  != r['prev_Marca']:  return 'C_brand'
    if r['vol']    != r['prev_vol']:    return 'C_vol'
    if r['pack']   != r['prev_pack']:   return 'C_pack'
    if r['Envase'] != r['prev_Envase']: return 'C_envase'
    return 'C0_self'

ch['chtype'] = ch.apply(classify_edge, axis=1)

# --- Temporal split: H1 = Jan-Jun (prior), H2 = Jul-Dec (validation) ---------
SPLIT = pd.Timestamp('2025-07-01')
ch['half'] = np.where(ch['fecha'] < SPLIT, 'H1', 'H2')
h1, h2 = ch[ch['half'] == 'H1'], ch[ch['half'] == 'H2']

# --- Goodness of Fit: fit 4 families by MLE, compare by AIC + KS + CvM --------
# stats.{gamma, weibull_min, lognorm, invgauss}.fit(s, floc=0)  -> MLE
# stats.kstest(s, dist.cdf, args=params)                        -> KS test
# stats.cramervonmises(s, dist.cdf, args=params)                -> CvM test
DIST_CATALOG = {'Gamma': stats.gamma, 'Weibull': stats.weibull_min,
                'LogNormal': stats.lognorm, 'InvGauss': stats.invgauss}
EDGE_TYPES = ['C_pack', 'C_brand', 'C_vol', 'C0_self', 'C_envase']

def gof_table(sample):
    rows = []
    for name, dist in DIST_CATALOG.items():
        params = dist.fit(sample, floc=0)              # MLE, location fixed at 0
        ll  = dist.logpdf(sample, *params).sum()
        aic = -2 * ll + 2 * 2                           # k = 2 free parameters
        ks_d, ks_p = stats.kstest(sample, dist.cdf, args=params)
        cvm = stats.cramervonmises(sample, dist.cdf, args=params)
        rows.append({'Dist': name, 'AIC': aic, 'KS_D': ks_d, 'KS_p': ks_p,
                     'CvM_W2': cvm.statistic, 'CvM_p': cvm.pvalue, 'params': params})
    return sorted(rows, key=lambda r: r['AIC'])          # best = lowest AIC

best_family = {}
for g in EDGE_TYPES:
    s = h1[h1['chtype'] == g]['hours'].values
    if len(s) >= 10:
        best_family[g] = gof_table(s)[0]                # winner per edge type
```

A.2 Bayesian conjugate updating and stationarity tests

```
"""
```

```
Appendix A.2 -- Bayesian conjugate updating and stationarity tests.
Source: notebooks/07_statistical_analysis.ipynb (Sections 2.3-2.6).

Gamma-Exponential conjugate model:
    X_i | lambda ~ Exp(lambda),   lambda ~ Gamma(a0, rate=b0)
    posterior after n obs:  lambda | x ~ Gamma(a0 + n, b0 + sum x_i)
    predictive duration:    E[X] = E[1/lambda] = b / (a - 1)   (a > 1)
Quantiles of lambda -> 1/quantile give the 95% interval for E[X]
(implemented with scipy.stats.gamma.ppf).
"""
import numpy as np
from scipy import stats


def sequential_bayes(h1_obs, h2_obs, prior='informative'):
    """Sequential Gamma-Exponential update; returns posterior mean path + 95% CI.

    informative : a0 = n_H1, b0 = sum_H1 x   (H1 enters as a pseudo-sample)
    uniform     : a0 = 1,    b0 = 0          (Jeffreys-type, learns only from H2)
    """
    h1_obs = np.asarray(h1_obs, float)
    h2_obs = np.asarray(h2_obs, float)
    if prior == 'informative':
        a0, b0 = len(h1_obs), h1_obs.sum()
    elif prior == 'uniform':
        a0, b0 = 1.0, 0.0
    else:
        raise ValueError("prior must be 'informative' or 'uniform'")

    a_seq, b_seq = [a0], [b0]
    for x in h2_obs:                        # one observation at a time
        a_seq.append(a_seq[-1] + 1)
        b_seq.append(b_seq[-1] + x)
    a_arr, b_arr = np.array(a_seq, float), np.array(b_seq, float)
    mean_seq = np.where(a_arr > 1, b_arr / (a_arr - 1), np.nan)

    ci_lo, ci_hi = [], []
    for a, b in zip(a_arr, b_arr):
        if a <= 1 or b <= 0:
            ci_lo.append(np.nan); ci_hi.append(np.nan); continue
        lam_hi = stats.gamma.ppf(0.975, a, scale=1 / b)   # high lambda -> low E[X]
        lam_lo = stats.gamma.ppf(0.025, a, scale=1 / b)
        ci_lo.append(1 / lam_hi); ci_hi.append(1 / lam_lo)
    return a_arr, b_arr, mean_seq, np.array(ci_lo), np.array(ci_hi)


def stationarity_tests(s1, s2):
    """Two-sample H1-vs-H2 distribution-equality tests.

    KS  : stats.ks_2samp            (supremum of |F1 - F2|)
    AD  : stats.anderson_ksamp      (Anderson-Darling, tail-weighted)
    CvM : stats.cramervonmises_2samp(Cramer-von Mises, uniform weight)
    p > 0.05  ->  do not reject H0: F_H1 = F_H2  (process is stationary).
    """
    ks  = stats.ks_2samp(s1, s2)
    ad  = stats.anderson_ksamp([s1, s2])
    cvm = stats.cramervonmises_2samp(s1, s2)
    return {
        'KS_D': ks.statistic,  'KS_p': ks.pvalue,
        'AD_stat': ad.statistic, 'AD_sig_level': ad.significance_level,
        'CvM_stat': cvm.statistic, 'CvM_p': cvm.pvalue,
    }


def empirical_bayes_prior(ch, edge_col='edge', min_n=2):
    """Per-edge Gamma prior when n >= min_n; otherwise fall back to the
    global Gamma prior used as a hierarchical regulariser."""
    a_g, _, sc_g = stats.gamma.fit(ch['hours'].values, floc=0)   # global prior
    priors = {}
    for edge, grp in ch.groupby(edge_col):
        s = grp['hours'].values
        if len(s) >= min_n:
            a, _, sc = stats.gamma.fit(s, floc=0)
            priors[edge] = (a, sc, 'specific')
        else:
```

```
            priors[edge] = (a_g, sc_g, 'global')
    return priors, (a_g, sc_g)
```

A.3 Permutation tests for ordered alternatives

```
"""
Appendix A.3 -- Permutation tests for ordered alternatives.
Source: notebooks/07_statistical_analysis.ipynb (Section 3).

Only numpy + scipy.stats.rankdata are needed: the null distribution is built
by shuffling the group labels (10,000 permutations), so no parametric
assumption on the changeover-duration distribution is required.
"""
import numpy as np
from scipy import stats
from itertools import combinations


def rank_means(groups):
    """Mean of the joint ranks within each group."""
    ranks = stats.rankdata(np.concatenate(groups))
    R, idx = [], 0
    for g in groups:
        R.append(ranks[idx:idx + len(g)].mean()); idx += len(g)
    return np.array(R)


def omnibus_rank_statistic(groups):
    """Q = sum_i n_i (R_i - R_bar)^2  : global equality (any difference)."""
    R = rank_means(groups)
    sizes = np.array([len(g) for g in groups])
    R_bar = np.average(R, weights=sizes)
    return np.sum(sizes * (R - R_bar) ** 2)


def T_statistic_k4(groups):
    """Ordered contrast for k = 4 groups: T = 3R4 + R3 - R2 - 3R1.
    Large T supports m_1 <= m_2 <= m_3 <= m_4."""
    R1, R2, R3, R4 = rank_means(groups)
    return 3 * R4 + R3 - R2 - 3 * R1


def pair_order_statistic(groups):
    """Two-group ordered statistic T_ij = R_j - R_i (large -> group i < group j)."""
    R1, R2 = rank_means(groups)
    return R2 - R1


def jt_statistic(groups):
    """Jonckheere-Terpstra statistic JT = sum_{i<j} U_ij for ordered groups."""
    k, jt = len(groups), 0.0
    for i in range(k):
        for j in range(i + 1, k):
            for x in groups[i]:
                jt += np.sum(groups[j] > x)
    return jt


def permutation_test(groups, stat_fn, n_perm=10_000, seed=42, alternative='greater'):
    """Label-permutation test: returns (observed stat, null sample, p-value)."""
    rng = np.random.default_rng(seed)
    sizes = [len(g) for g in groups]
    pooled = np.concatenate(groups)
    T_obs = stat_fn(groups)
    T_null = np.empty(n_perm)
    for b in range(n_perm):
        p = rng.permutation(pooled)
        pg, idx = [], 0
        for n_i in sizes:
            pg.append(p[idx:idx + n_i]); idx += n_i
        T_null[b] = stat_fn(pg)
    if alternative == 'greater':
```

```
        p_val = (np.sum(T_null >= T_obs) + 1) / (n_perm + 1)
    else:  # two-sided around the null mean
        c = T_null.mean()
        p_val = (np.sum(np.abs(T_null - c) >= abs(T_obs - c)) + 1) / (n_perm + 1)
    return T_obs, T_null, p_val


# ---- Application A (edge types, k=4) and B (lines, k=3) ----------------------
# groups_type = [hours[chtype==g] for g in ['C_pack','C_brand','CO_self','C_envase']]
# Q_obs, _, p_omni = permutation_test(groups_type, omnibus_rank_statistic)
# T_obs, _, p_ord  = permutation_test(groups_type, T_statistic_k4, seed=43)
#
# Bonferroni-corrected ordered post-hoc over the C(4,2)=6 pairs:
# for (i, j) in combinations(range(4), 2):
#     permutation_test([groups_type[i], groups_type[j]], pair_order_statistic)
#
# groups_line = [hours[line==l] for l in [17, 19, 14]]
# JT_obs, _, p_line = permutation_test(groups_line, jt_statistic)
# post-hoc lines: stats.mannwhitneyu(g_a, g_b, alternative='less'), Bonferroni 0.05/3
```